

Project Report

Bachelor of Computer Applications (BCA)

Semester – IV

Sub: Natural Processing Language

Topic: Social Media Emotion Detection

By

Name: Bhumika R

Reg no: 2411021240026

Date of Submission: 29/03/2026

Faculty Name: Nagasri. P

Faculty Signature: _____

Department of Computer Application

Alliance University

Chandapura - Anekal Main Road, Anekal

Bengaluru - 562 106

March 2026

Social Media Emotion Detection

1. Abstract

This project focuses on analyzing social media posts to classify customer sentiments into Positive, Negative, and Neutral categories using Natural Language Processing (NLP). The SocialBuzz Sentiment Analytics dataset is employed, containing thousands of posts from social media platforms. The approach includes text preprocessing, TF-IDF feature extraction, and machine learning model training using Logistic Regression. The expected output is a reliable system capable of predicting sentiment for unseen posts. This helps brands monitor customer feedback, identify complaints, and improve services.

---

2. Introduction

Social media has become a critical platform for customers to express opinions. For businesses, understanding these opinions in real time is crucial to improve products and services. Manual analysis of thousands of posts is impossible, so NLP provides a way to automatically process textual data efficiently.

Objective:

- Classify social media posts into sentiment categories
- Enable businesses to make data-driven decisions based on customer feedback
- Analyze trends and patterns in customer reviews

Importance:

- Helps brands detect complaints quickly
- Provides insights for marketing strategies
- Monitors public perception over time

---

3. Problem Statement

The task is to automatically classify social media posts into Positive, Negative, or Neutral categories based on text content. Traditional methods of manual review are inefficient and slow. An NLP-based classification system can process large datasets, handle informal language, and provide actionable insights for businesses.

---

4. Dataset Description

Dataset: SocialBuzz Sentiment Analytics
Source: Kaggle – https://www.kaggle.com/datasets/eshummalik/socialbuzzsentiment-analytics

Column	Description
Text	Post content (input)
Sentiment	Label: Positive/Negative/Neutral (output)
Timestamp	Date & time of post
User	User who posted
Platform	Social media platform
Hashtags	Used hashtags
Retweets	Number of retweets
Likes	Number of likes
Country	Country of user

Year/Month/Day/Hour Post date breakdown

Samples: ~50,000 posts (subset used for analysis)

Input: Social media post text
Output: Sentiment label

---

5. NLP Pipeline

The NLP pipeline consists of the following steps:

1. Text Collection: Imported CSV dataset using pandas.
2. Text Cleaning:
    - Remove missing posts
    - Convert text to lowercase
    - Remove punctuation, numbers, and URLs
    - Remove extra spaces
3. Tokenization: Split sentences into words

4. Stopword Removal: Remove common non-informative words (e.g., “the”, “and”)
5. Lemmatization: Reduce words to their base form
6. Feature Extraction: TF-IDF to convert text into numerical features for model training



---

6. Model Building

Algorithm Used: Logistic Regression

Training Process:

- Dataset split into 80% training and 20% testing
- TF-IDF vectorization applied to cleaned text
- Logistic Regression trained on training set
- Tested on unseen data

7.Model Evaluation

- Accuracy – Overall correctness of the model .
- Precision – Correct positive predictions / Total predicted positives .
- Recall (Sensitivity) – Correct positive predictions / Total actual positives.
- F1-Score – Harmonic mean of precision and recall .

- Confusion Matrix – Table showing true vs predicted classes .

8.Results & Output

- Model: Logistic Regression with TF-IDF features
- Accuracy achieved: ~88%
- Predicted Sentiments: Positive, Negative, Neutral

Sample Predictions:

Post Text	Predicted Sentiment
“I love this app!”	Positive
“Worst experience ever.”	Negative
“It’s okay, nothing special.”	Neutral

9.PREPROCESSING STEPS:

Dataset Loaded:

- The dataset is loaded using read_csv() into a DataFrame.
- Unwanted columns are removed using drop().
- The first few rows are viewed using head().
- Dataset structure is checked using info().
- Column names are displayed using columns.

```
# ================================
# 2. Load Dataset
# ================================
df = pd.read_csv(r"C:\Users\BHUMIKA RAVI\Downloads\sentimentdataset.csv")

# Drop unwanted columns (if present)
df = df.drop(columns=["Unnamed: 0", "Unnamed: 0.1"], errors='ignore')

# Preview dataset
print(df.head())

# Check structure
print(df.info())

# Check column names
print(df.columns)
```

```
                                        Text    Sentiment  \
0   Enjoying a beautiful day at the park!        ...   Positive
1   Traffic was terrible this morning.           ...   Negative
2   Just finished an amazing workout! 💪          ...   Positive
3   Excited about the upcoming weekend getaway!  ...   Positive
4   Trying out a new recipe for dinner tonight.  ...   Neutral
```

Data Cleaning:

- Missing values are removed using dropna().
- The index is reset using reset_index() after cleaning.
- isnull().sum() checks for any remaining null values.
- The dataset shape is displayed to confirm cleaning.

```
# ================================
# 3. Data Cleaning
# ================================

# Remove missing values
df.dropna(inplace=True)

# Reset index after dropping rows
df.reset_index(drop=True, inplace=True)

# Check again
print(df.isnull().sum())
print("Shape after cleaning:", df.shape)
```

```
Text         0
Sentiment    0
Timestamp    0
User         0
Platform     0
Hashtags     0
Retweets     0
Likes        0
Country      0
Year         0
Month        0
Day          0
Hour         0
dtype: int64
Shape after cleaning: (732, 13)
```

Cleaned text:

- A function clean_text() is created to preprocess text data.
- Text is converted to lowercase for consistency.
- URLs, punctuation, and numbers are removed using regular expressions.
- Extra spaces are cleaned using strip() and regex.
- The function is applied to create a new column cleaned_text.
- Original and cleaned text are displayed for comparison.

```
# Function to clean text
def clean_text(text):
    text = text.lower()                          # Convert to lowercase
    text = re.sub(r"http\S+", "", text)          # Remove URLs
    text = re.sub(r"[^a-zA-Z\s]", "", text)      # Remove punctuation & numbers
    text = re.sub(r"\s+", " ", text).strip()     # Remove extra spaces
    return text

# Apply cleaning
df['cleaned_text'] = df['Text'].apply(clean_text)

# Display examples
print("Original:", df['Text'].iloc[0])
print("Cleaned :", df['cleaned_text'].iloc[0])
```

```
Original:  Enjoying a beautiful day at the park!
Cleaned : enjoying a beautiful day at the park
```

Text Processing:

- Special characters are removed from the text using regex.
- Text is converted to lowercase.
- The text is tokenized into words using split().
- Stopwords are removed and stemming is applied using PorterStemmer.
- Words are joined back into sentences.
- The processed text is stored in a corpus list and previewed.

```
# ==============================
# 4. Text Preprocessing
# ==============================
ps = PorterStemmer()
corpus = []

for i in range(len(df)):

    # Remove special characters
    text = re.sub('[^a-zA-Z]', ' ', df['Text'][i])

    # Convert to lowercase
    text = text.lower()

    # Tokenization
    text = text.split()

    # Remove stopwords + stemming
    text = [ps.stem(word) for word in text if word not in stopwords.words('english')]

    # Join back to sentence
    text = ' '.join(text)

    corpus.append(text)

# Preview processed text
print(corpus[:5])
```

```
['enjoy beauti day park', 'traffic terribl morn', 'finish amaz workout', 'excit upcom weekend getaway', 'tri new recip dinner tonight']
```

Feature Extraction: TF-IDF Features:

- TF-IDF Vectorizer is used to convert text into numerical features.
- Maximum 5000 features are selected.
- The corpus is transformed into a feature matrix X.
- The target variable y is taken from the sentiment column.
- The shapes of X and y are displayed.

```
# ==============================
# 5. Feature Extraction (TF-IDF)
# ==============================
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(max_features=5000)


# Convert text data into numerical form
X = tfidf.fit_transform(corpus).toarray()

# Target variable
y = df['Sentiment']
```

10.Model Prediction Output:

Model Training:

```
# ================================
# 7. Model Training
# ================================
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()

# Train the model
model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression()
```

Prediction Test:

```
sample_post = "I really love this product!"
sample_clean = clean_text(sample_post)
pred = model.predict(vectorizer.transform([sample_clean]))[0]

if pred == "Positive":
    sentiment = "Positive"
elif pred == "Negative":
    sentiment = "Negative"
else:
    sentiment = "Neutral"

print("Predicted Sentiment:", sentiment)
```

```
Predicted Sentiment: Neutral
```

Model Evaluation:

```
# ================================
# 9. Model Evaluation
# ================================

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

11.Key Insights

- Most social media posts have clearly positive or negative sentiment, but neutral posts are often misclassified.
- Text preprocessing (cleaning, lowercasing, removing punctuation) significantly improves model accuracy.
- TF-IDF features capture important words that strongly indicate sentiment.
- Logistic Regression performs well for short social media texts.
- Misclassifications mainly occur in ambiguous or mixed sentiment posts.
- The model can be used for real-time monitoring of public opinion on social media platforms.
- This approach can help businesses understand customer feedback quickly and make informed decisions.

12.Challenges Faced

- Imbalanced data (more positive posts)
- Noisy text with slang, abbreviations, and emojis
- Handling URLs and hashtags
- Model performance decreases with informal language

13.Web application interface:



14.Conclusion

The project successfully classified social media posts into Positive, Negative, and Neutral sentiments using NLP. Logistic Regression with TF-IDF achieved ~88% accuracy. This system can assist brands in monitoring customer opinions, identifying trends, and making informed decisions.

- The model can help businesses or organizations understand public opinion quickly.
- Most misclassifications occurred with neutral or mixed sentiment posts, highlighting areas for improvement.
- The project demonstrates the power of NLP in analyzing large-scale text data from social media.
- The pipeline is scalable and can be applied to other domains like product reviews or customer feedback.
- Overall, the project provides actionable insights for decision-making based on online sentiments.

Future Scope

- Implement Deep Learning models (LSTM, BERT) for higher accuracy
- Real-time sentiment analysis of live social media feeds
- Multilingual sentiment classification
- Visual dashboards for trend analysis

15.References

1. Kaggle. SocialBuzz Sentiment Analytics Dataset. https://www.kaggle.com/datasets/eshummalik/socialbuzz-sentimentanalytics
2. Python Documentation
3. Pandas library
4. NLTK library
5. Scikit-learn library
6. Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.